

# Compresor de texto basado en Huffman

Geor Sebastián Gómez Correa

Tomás Camilo García López

Eduardo Castellanos Márquez



## ¿De qué trata el proyecto?

Queremos reducir el tamaño de un texto. ¿Cómo? (Huffman)

- Asignando bits más cortos a los caracteres más frecuentes.
- Motivación: almacenamiento, transmisión, eficiencia, uso de estructuras de datos.

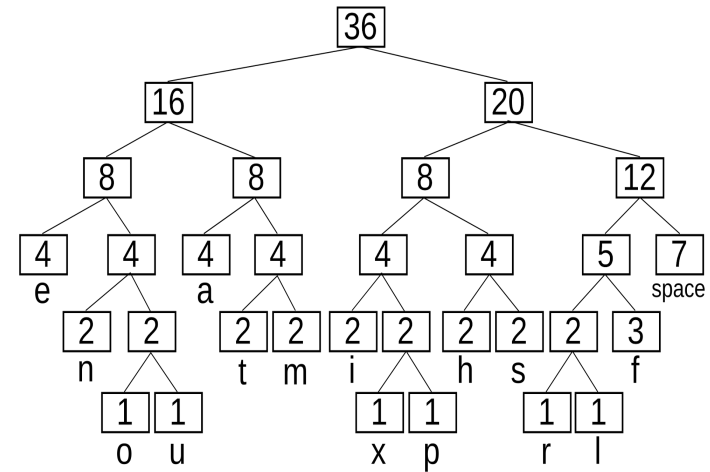
# Estructuras

## Montículo / Cola con Prioridad

- Inserta nodos según frecuencia.
- Permite construir el árbol de Huffman de forma eficiente.

## Árbol Binario

- Representa los códigos finales.
- Cada hoja es un carácter.





## Matriz Dispersa (comparador)

- Tamaño original
- Tamaño comprimido

## Diccionario (mapa)

- Para contar frecuencias.
- Rápido acceso  $O(1)$ .



# Características del proyecto

Lenguaje: 100% Python

Entorno de desarrollo: Visual Studio Code

Repositorio: Github

Porcentaje de IA: Código: 75%, Arquitectura: 40%



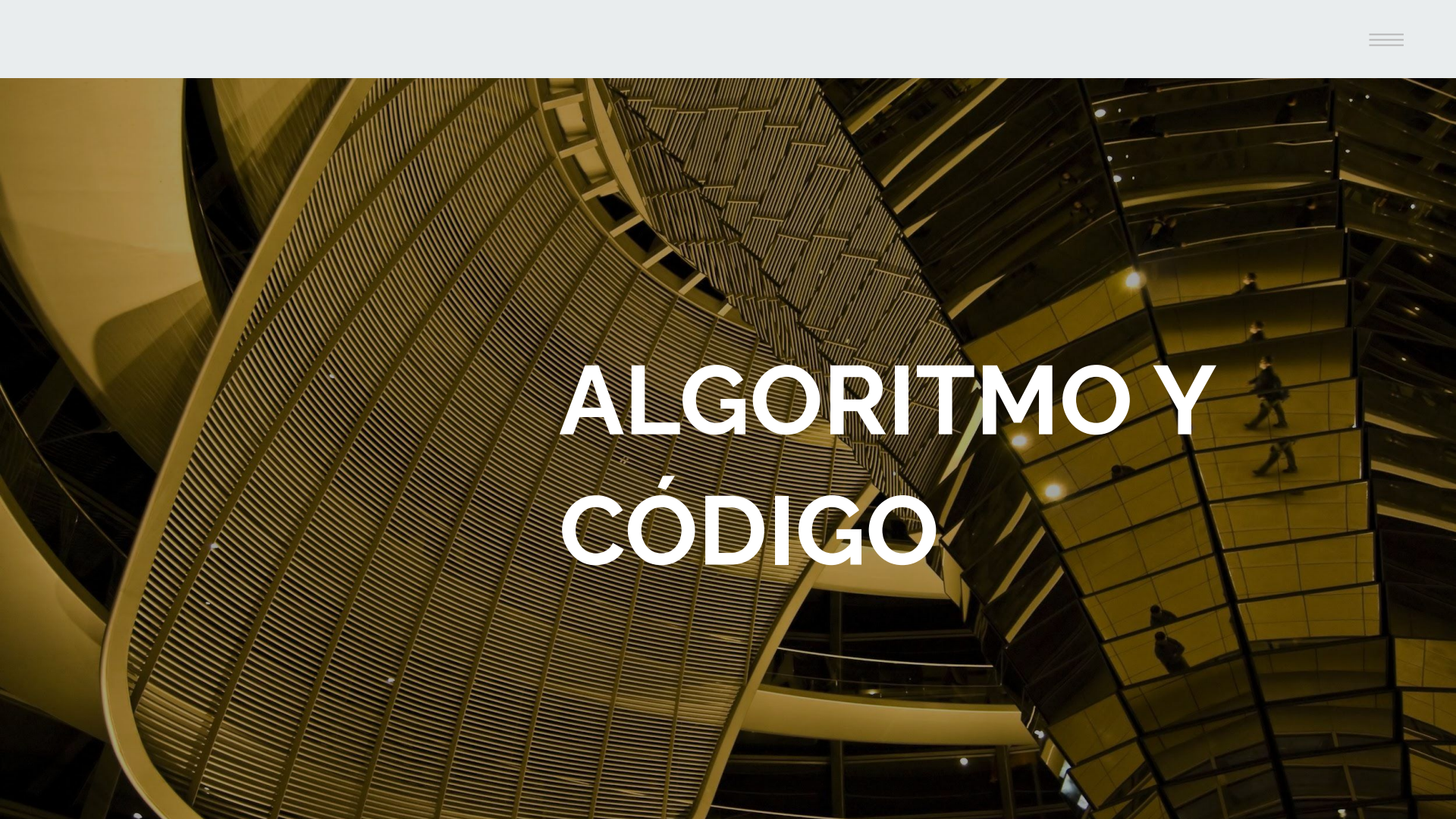
# Características del proyecto

Lenguaje: 100% Python

Entorno de desarrollo: Visual Studio Code

Repositorio: Github

Porcentaje de IA: Código: 75%, Arquitectura: 40%



# ALGORITMO Y CÓDIGO





## Conclusiones

**Huffman reduce el tamaño del texto** aprovechando la frecuencia desigual de los caracteres.

El algoritmo garantiza **códigos prefijo**, así que la descompresión siempre es correcta.

Las estructuras usadas (**heap, árbol binario y hash**) hacen que el algoritmo sea **eficiente y determinístico**.

La comparación final demuestra que el método **sí logra compresión real** y es fácil de integrar en sistemas más grandes.



# Gracias.

